BSD Sockets

Sockets

- O metodă de comunicare între componente software
- Descrisă prin descriptori de fișier, capete ale canalului de comunicare

- Mai multe tipuri:
  - comunicare locală
  - comunicare la distanță, prin rețea

Network Socket

= O primitivă reprezentând un punct de comunicație prin rețea

Definit de:
- Protocolul de comunicare folosit (TCP, UDP)
- Adresa de socket (IP, port)

Tipuri:
- Stream sockets (folosesc TCP protocol)
- Datagram sockets (folosesc protocolul UDP)
- Raw IP sockets (de nivel scăzut)

Sockets si programare

Programele pot folosi sockets print-un socket API

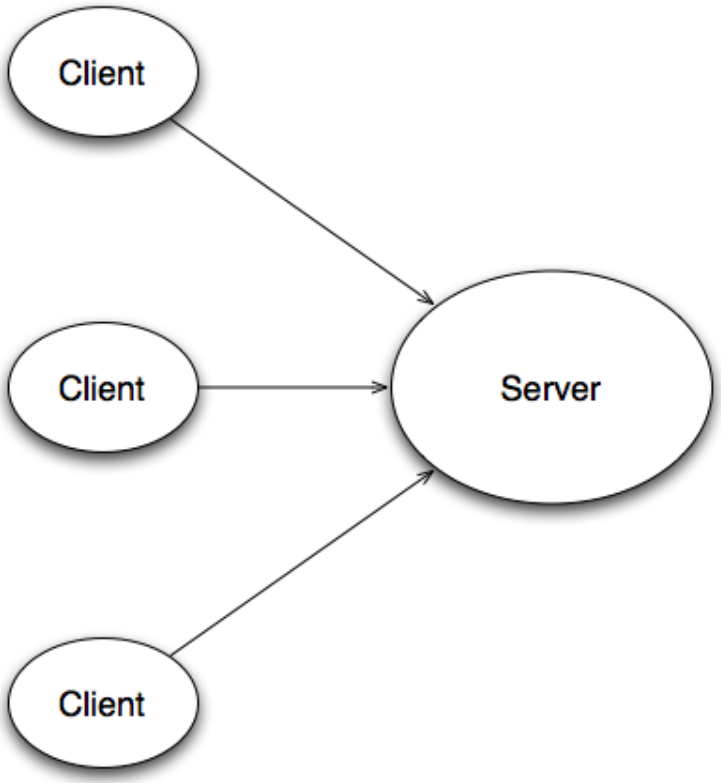
- disponibile in toate platformele software moderne
- accessibile din majoritatea sistemelor de operare moderne
- Se foloseste paradigma de comunicare client-server

BSD Sockets API defineste:

- Network sockets (de obicei prin stiva de protocoale TCP/IP)
- UNIX Domain sockets (comunicare locala)

Client-server

- Server: oferă un set of *servicii*
- Client: folosește serviciile
- Clientul inițiază comunicarea
- De obicei, clienții sunt mai mici decât serverele (dar nu neapărat)

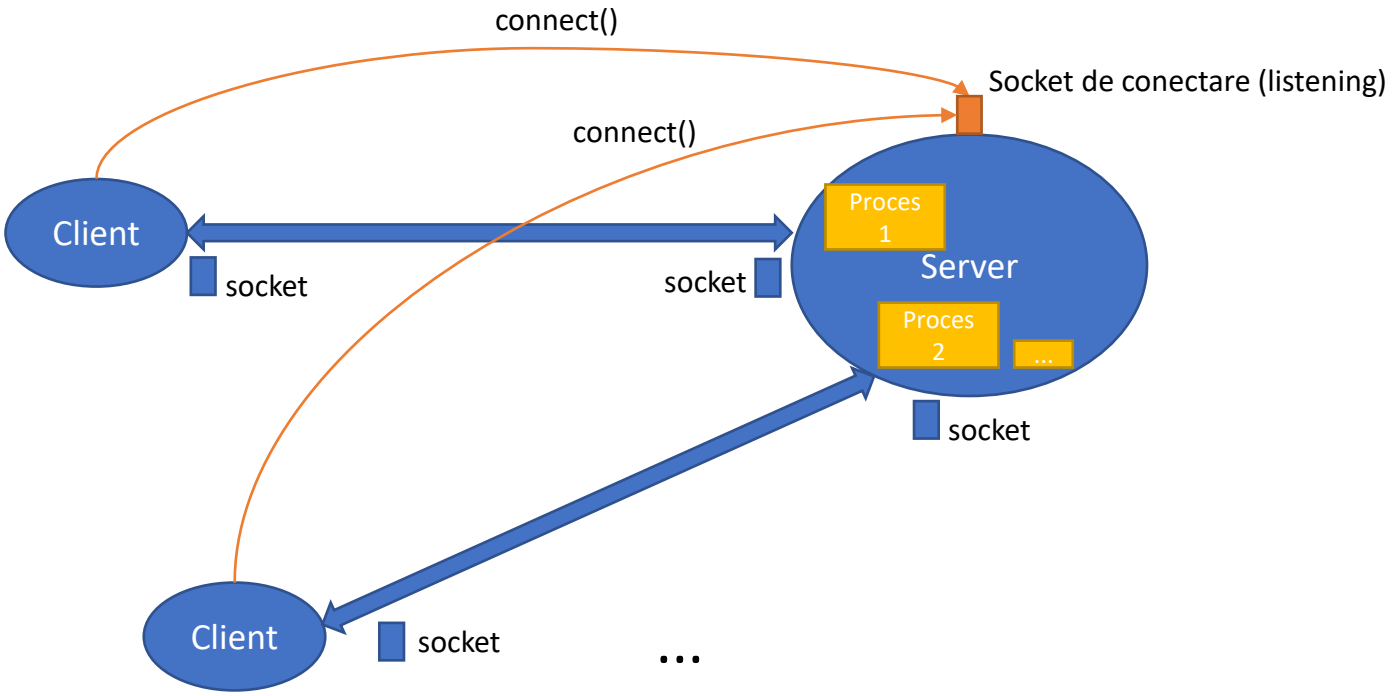


(c) 2008, Dan C. Cosma

TCP/IP. Paradigme de comunicare

- Bazată pe conexiune (connection-oriented)
  - La inceputul comunicării se stabilește un canal permanent între client și server
  - Se bazează pe protocolul TCP (Transmission Control Protocol)

- Fără conexiune (connectionless)
  - La fiecare trimitere de date se precizează destinația (și la recepție se precizează sursa de interes)
  - Se bazează pe protocolul UDP (User Datagram Protocol)



Comunicarea în conexiuni TCP

BSD Sockets

C headers:

<sys/socket.h>

Core functionality

<netinet/in.h>

internet, address and protocol families

<sys/un.h>

Local address family, not used on networks

<arpa/inet.h>

Functions for manipulating IP addresses

<netdb.h>

Functions name resolution (e.g. DNS)

BSD Sockets – main functions

int socket(int domain, int type, int protocol);

Client and server side. Creates a socket. Arguments:

- domain: AF_INET (IPv4), AF_INET6 (IPv6), AF_UNIX (non-network)
- type: SOCK_STREAM (TCP), SOCK_DGRAM (UDP), etc.
- protocol: IPPROTO_TCP, IP_PROTO_UDP

Returns: a new file descriptor representing the socket, or -1 on error

int bind(int sockfd, const struct sockaddr *my_addr, socklen_t addrlen);

Client and server side. Binds a socket to an address (IP and port). Mandatory for server-side sockets. Clients usually don't call this function except for the rare cases when a specific port is needed at the client end (e.g. because of a firewall restriction on outgoing ports)

int listen(int sockfd, int backlog);

Server side. Prepares a socket for incoming connections. Necessary only for stream (TCP) sockets.

BSD Sockets – main functions

int accept(int sockfd, struct sockaddr *cliaddr, socklen_t *addrlen);

Server side. Waits for an incoming connection. Returns a new socket when the connection is established. Only required for stream sockets (TCP).

The new socket:

- will be bound to the same port
- represents the communication endpoint with the client that initiated the connection

int connect(int sockfd, const struct sockaddr *serv_addr, socklen_t addrlen);

Client side. Connects to a server specified by IP address and port. If the client-side socket wasn't already bound to a port, an *ephemeral port* will be created and bound for this endpoint.

Used mainly for connection-oriented protocols (TCP)

For connectionless protocols, connect only sets the *default* destination address (to use with send(), as opposed to sendto())

Helper functions

```
struct hostent *gethostbyname(const char *name);
struct hostent *gethostbyaddr(const void *addr, int len, int type);
```

Server and client side. Support functions for translating to/from host names using DNS or other resolving methods (such as local /etc/hosts files)

Sending data

```
ssize_t write(int fd, const void *buf, size_t count);
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
```

Sends data through a connected socket. If the protocol is connectionless, the default destination is used, as set with connect().

```
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,
                const struct sockaddr *dest_addr, socklen_t addrlen);
```

Send data to a specific address. If the socket is connection-oriented (TCP), the given destination address is ignored.

Receiving data

```
ssize_t read(int fd, void *buf, size_t count);
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
```

Receives data from a connection-oriented socket.

```
ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,
                 struct sockaddr *src_addr, socklen_t *addrlen);
```

Receives data, usually from a socket. If not null, the source address is filled-in with the address of the sender.

Tipuri de servere

- Servere iterative
  - Pot servi doar un client la un moment dat
- Servere concurente
  - Pot servi mai mulți clienți simultan
  - Folosesc primitive specifice sistemelor concurente (de exemplu procese, threads)

Exemplu de server iterativ

```
int sockfd, newsockfd;
if ((sockfd=socket(...)) < 0)  {
    printf ("error ..."); exit(1);
  }

if (bind(sockfd,...) < 0)  {
    printf ("error ..."); exit(1);
  }

if (listen(sockfd,5) < 0)  {
    printf ("error ..."); exit(1);
  }



for (;;)
{
 newsockfd = accept(sockfd, ...);
 if (newsockfd < 0)  {
     printf ("error ..."); exit(1);
   }

  process(newsockfd);
      /*handle the client*/

  close (newsockfd);
}
```

Exemplu de server concurent

```
int sockfd, newsockfd;

if ((sockfd=socket(...)) < 0){
     printf ("error ...");
     exit(1);
  }

if (bind(sockfd,...) < 0){
     printf ("error ...");
     exit(1);
  }

if (listen(sockfd,5) < 0){
     printf ("error ...");
     exit(1);}
  }

for (;;)
{
 newsockfd=accept(sockfd, ...);
 if (newsockfd < 0) {
     printf ("error ...");
     exit(1);
   }

 if (fork()==0)   {
     close(sockfd);
     process(newsockfd);
       /*handle the client*/
     exit(0);
   }

 close (newsockfd);
}
```